

# Unity C# 脚本基础

目标：用 Unity 和 C# 完成一个 2D Pong 原型，理解 GameObject、Component、生命周期函数、输入、碰撞、事件通信和调试。

## 一、引擎基础知识

### 1. Unity 编辑器复习

- 编辑器界面概览：Hierarchy、Scene、Game、Inspector、Project、Console。
- GameObject：场景中的对象，本身只是容器。真正的行为来自 Component。
- Component：Transform、SpriteRenderer、Rigidbody2D、Collider2D、脚本等都属于组件。
- Scene：当前关卡或界面。Prefab：可复用的 GameObject 模板。

**to do 0: 创建 2D 场景，新建空物体 Pong，新建子物体 Ball 和 Paddle，并为 Ball 添加 SpriteRenderer。**

### 2. C# 脚本与 MonoBehaviour

- Unity 脚本通常是一个继承 MonoBehaviour 的 C# 类。文件名必须和类名一致。
- 把脚本拖到 GameObject 上，脚本就变成这个物体的一个 Component。
- C# 是强类型语言：变量声明时通常要写清楚类型，例如 int、float、string、bool。
- Unity 常用类型：Vector2、Vector3、Transform、GameObject、Rigidbody2D、Collider2D。

```
using UnityEngine;

public class BallController : MonoBehaviour
{
    void Start()
    {
        Debug.Log("Ball ready");
    }
}
```

## 二、C# 基础语法

### 1. 脚本创建与结构

- `Awake()`：脚本实例加载时调用，常用于缓存组件引用。
- `Start()`：第一次 Update 前调用一次，常用于初始化游戏状态。
- `Update()`：每帧调用一次，间隔不稳定，常用于读取输入、非物理逻辑。
- `FixedUpdate()`：固定时间间隔调用，常用于 Rigidbody 相关的物理移动。

to do 1: 在 Console 输出 Awake、Start、Update、FixedUpdate，观察执行顺序。

to do 2: 熟悉 2D Scene 视图，尝试移动、旋转、缩放 Ball。

to do 3: 读取 transform.position 并更新，实现 Ball 自动移动。

```
using UnityEngine;

public class MoveRight : MonoBehaviour
{
    [SerializeField] float speed = 3f;

    void Update()
    {
        Vector3 pos = transform.position;
        pos.x += speed * Time.deltaTime;
        transform.position = pos;
    }
}
```

## 2. 变量与常量

- 声明变量：int score = 0;、float speed = 5f;
- public 变量会暴露在 Inspector，但初学阶段更推荐 [SerializeField] private
- 常量：const int MaxScore = 10;。只读字段：readonly float gravity;
- 命名规范：类名 PascalCase，方法名 PascalCase，局部变量和私有字段常用 camelCase。

to do 4: 声明 speed 变量，并用 [SerializeField] 暴露到 Inspector。

## 3. 运算符

- 算术运算符：+ - \* / %
- 赋值运算符：= += -= \*= /=
- 比较运算符：== != < > <= >=。先用它们写条件判断。
- 逻辑运算符：&& || !，分别表示与、或、非。

to do 5: 在 Start() 里尝试各种运算，并用 Debug.Log 输出结果。

to do 6: 使用 speed 变量更新 Ball 的位置。

## 4. 基本输入处理

- 最简单的键盘输入：Input.GetKey(KeyCode.W) 持续按下。
- 刚按下：Input.GetKeyDown(KeyCode.Space)。刚松开：Input.GetKeyUp(KeyCode.Space)。
- 连续轴输入：Input.GetAxisRaw("Vertical")，默认可读取 W/S 或方向键。

to do 7: 在 Project Settings/Input Manager 中查看 Horizontal 和 Vertical 轴。

to do 8: 新建 PaddleController.cs，在合适的函数里输出 up button pressed 和 down button pressed。

to do 9: 实现按 W 和 S 移动 Paddle。提示：每帧读取输入，再更新 transform.position。

```
using UnityEngine;

public class PaddleController : MonoBehaviour
{
    [SerializeField] float speed = 6f;
    [SerializeField] float limitY = 4f;

    void Update()
    {
        float input = Input.GetAxisRaw("Vertical");
        Vector3 pos = transform.position;
        pos.y += input * speed * Time.deltaTime;
        pos.y = Mathf.Clamp(pos.y, -limitY, limitY);
        transform.position = pos;
    }
}
```

## 三、编程概念

### 1. 函数

- 定义函数: `ReturnType FunctionName(ParameterType parameter)`。
- 无返回值使用 `void`, 有返回值使用 `return`。
- 变量作用域: 在函数内声明的变量只在函数内可用; 字段可被同一个类中的多个函数访问。

```
float GetMoveAmount(float input, float speed)
{
    return input * speed * Time.deltaTime;
}
```

### 2. 数组、List 和 Dictionary

- 数组: `int[] scores = {0, 0};`
- List: `List<GameObject> enemies = new List<GameObject>();`, 长度可变。
- Dictionary: `Dictionary<string, int> scoreMap`, 通过 key 访问 value。
- 遍历: `for`、`foreach`。

### 3. 控制流

- 条件语句: `if / else if / else`
- 循环语句: `for`、`while`、`foreach`
- 中断或跳过: `break`、`continue`

```

if (score >= 10)
{
    Debug.Log("Game Over");
}
else
{
    Debug.Log("Keep playing");
}

```

## 4. 组件、碰撞与触发

- 物理移动通常需要 Rigidbody2D，碰撞检测需要 Collider2D。
- 普通碰撞回调：OnCollisionEnter2D(Collision2D collision)。
- 触发器回调：OnTriggerEnter2D(Collider2D other)，需要勾选 Is Trigger。
- 用 GetComponent<T>() 获取同一个 GameObject 上的组件。

**to do 10:** 完善场地，添加 Collider2D，限定 Paddle 范围，并让 Ball 与边界反弹。

```

Rigidbody2D rb;

void Awake()
{
    rb = GetComponent<Rigidbody2D>();
}

void FixedUpdate()
{
    rb.velocity = new Vector2(3f, 2f);
}

```

# 四、事件与脚本通信

## 1. Unity 中的事件思路

- Godot 用 signal 解耦节点；Unity 常用 C# event、UnityEvent 或 Inspector 引用来通信。
- 简单项目中，优先把需要通信的对象通过 [SerializeField] 拖入 Inspector。
- 事件适合分数变化、游戏结束、按钮点击等“一件事发生了”的通知。

```

using System;
using UnityEngine;

public class ScoreZone : MonoBehaviour
{
    public event Action Scored;

    void OnTriggerEnter2D(Collider2D other)
    {
        Scored?.Invoke();
    }
}

```

to do 11: 用事件或 UnityEvent 实现重启、计分，并让 UI 能收到分数变化。

to do 12: 增加可玩性，包括随机方向发球、逐渐增速、根据击球位置改变反弹角度。

## 五、面向对象基础

---

### 1. 类的概念

- 一个 .cs 文件通常定义一个类，类中包含字段、属性、方法和事件。
- Unity 的脚本类继承 MonoBehaviour 后，才能挂到 GameObject 上并接收生命周期回调。

### 2. 继承与组合

- 继承语法：`public class PlayerPaddle : PaddleController`
- Unity 项目更常用组合：把多个小组件挂在同一个 GameObject 上，各自负责一件事。
- Prefab 可以保存一组组件和参数，是复用对象的主要方式。

## 六、调试

---

### 1. 基本调试技巧

- `Debug.Log()`、`Debug.LogWarning()`、`Debug.LogError()` 用于输出信息。
- 阅读 Console 的第一条错误，双击可跳到对应脚本行。
- 使用断点、Pause、Step 逐帧检查变量。
- `OnDrawGizmos()` 可在 Scene 视图里画辅助线和范围。

## 附：Godot 到 Unity 概念对照

| Godot                                | Unity                      | 说明                                          |
|--------------------------------------|----------------------------|---------------------------------------------|
| Node                                 | GameObject + Component     | Unity 的 GameObject 是容器，行为来自组件。              |
| Scene                                | Scene / Prefab             | Unity Scene 是关卡或界面，Prefab 是可复用对象模板。         |
| <code>_ready()</code>                | <code>Start()</code>       | 常用于初始化。Awake 会更早执行。                         |
| <code>_process(delta)</code>         | <code>Update()</code>      | 每帧逻辑，使用 <code>Time.deltaTime</code> 抵消帧率差异。 |
| <code>_physics_process(delta)</code> | <code>FixedUpdate()</code> | 固定间隔，适合 Rigidbody 物理相关逻辑。                   |
| Signal                               | C# event / UnityEvent      | 用于事件通知和脚本解耦。                                |